
PHY3075 — Modélisation numérique en physique

Nom : Danny Thou

Date de remise : 12 février 2026, 23h30

Matricule : 20278007

Devoir 2

1. Équation de chaleur

1.1. Solution exacte

1. L'énoncé nous donne la forme de la solution :

$$u(x,t) = \hat{u}(t) \sin(k\pi x) = T(t)X(x)$$

Par séparation de variables, on a que :

$$u_{xx} = X''(x)T(t)$$

$$u_t = X(x)T'(t)$$

Donc :

$$XT' = kX''T$$

$$\frac{X''}{X} = \frac{T'}{kT}$$

$$\frac{X''}{X} = -\lambda \quad \text{et} \quad \frac{T'}{kT} = -\lambda$$

On sait que $X(x) = \sin(k\pi x)$ à cause des conditions frontières

$$-(k\pi)^2 \sin(k\pi x) = -\lambda \sin(k\pi x)$$

$$\lambda = (k\pi)^2$$

Pour $T(t)$:

$$T' = -\lambda kT$$

$$\frac{T'}{T} = -k^2 \pi^2 k$$

$$\int \frac{T'}{T} dt = \int -k(k\pi)^2 dt$$

$$\ln(T) = -k(k\pi)^2 t + C$$

$$T = e^{-k(k\pi)^2 t + C}$$

$$T = D e^{-k(k\pi)^2 t} \quad \text{où } D: \text{ constante } = e^C$$

$$T = e^{-k(k\pi)^2 t} \quad \text{posons } D = 1$$

$$\text{Donc : } u(x,t) = e^{-k(k\pi)^2 t} \sin(k\pi x)$$

1.2. Algorithmme

1. On définit les pas de dérivations :

$$\Delta x = \frac{b - a}{n_x} \quad (1)$$

$$\alpha = \frac{\kappa \Delta t}{\Delta x^2} \leq 1/2 \quad (2)$$

$$\Delta t \leq \frac{\Delta x^2}{2\kappa} \quad (3)$$

$$n_t = \frac{t_{final}}{\Delta t} \quad (4)$$

où a,b sont les frontières des positions, t_{final} est le dernier temps et n est le nombre de points

2. On définit les conditions initiales :

- a) On crée un tableau $n_x \times n_t$ pour u
- b) On calcule les valeurs de x et sa condition initiale

$$x_i = a + i\Delta x \quad (5)$$

$$u_{i,0} = u(x, 0) \quad (6)$$

$$(7)$$

3. On fait une boucle temporelle pour j = 0 à $n_t - 1$:

- a) $t_j = j\Delta t$ et on ajoute la valeur à t_{liste}
- b) On calcule l'évolution de $u_{i,j}$ avec une boucle i=1 à n_x-1

$$u_{i,j+1} = u_{i,j} + \alpha(u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) \quad (8)$$

- c) On applique les conditions frontière :

$$u_{0,j+1} = u_{n_x,j+1} = 0 \quad (9)$$

4. On sort u

1.3. Résolution numérique

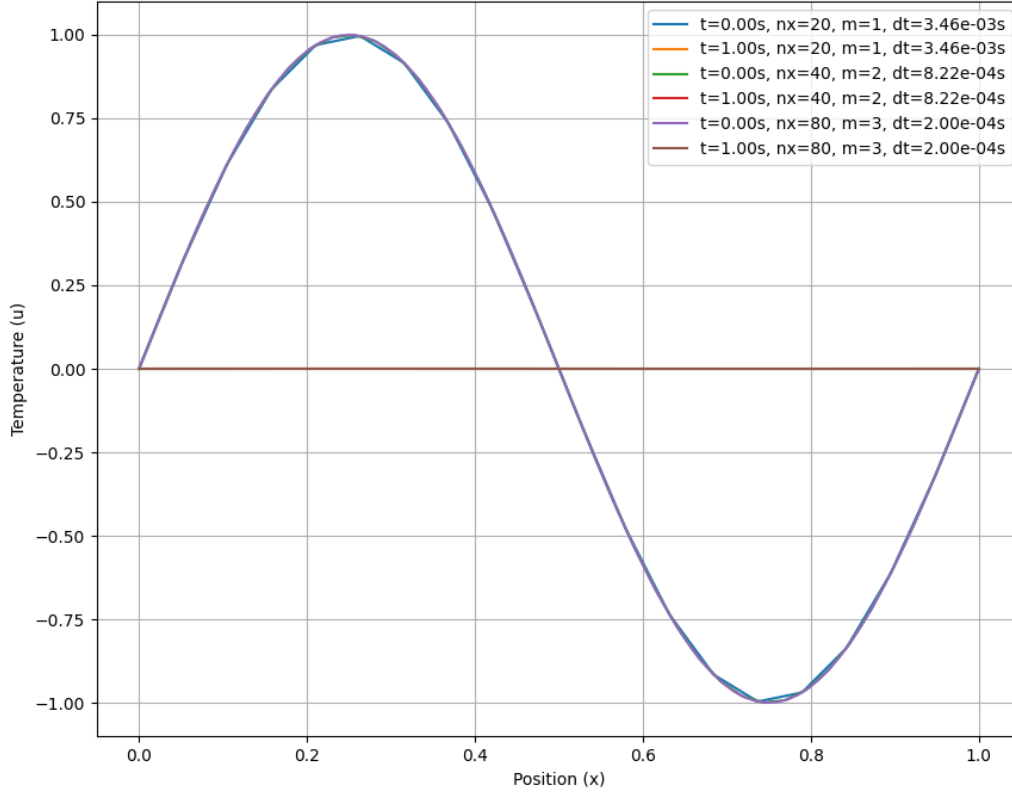


FIGURE 1 – Température dans la barre en fonction de la position à aux temps $t=0$ et à $t=1$

En nous référant à la figure 1, nous voyons que l'état initial et final de la répartition de la température ainsi que la valeur des paramètres n_x , m et Δt . Nous pouvons voir que nous retrouvons le bon comportement de la diffusion de chaleur, puisque la température atteint un équilibre à $t=1$. De plus, nous pouvons aussi voir l'effet de m sur l'allure de la courbe, avec 20 points ($m=1$), la courbe suit la même tendance, mais ne possède pas une courbe aussi lisse que pour $m=2$ et $m=3$. L'échelle du graphique ne nous permet pas de voir la différence de précision entre les différentes courbes. Le graphique ci-dessous montre l'erreur entre la solution analytique et la solution numérique :

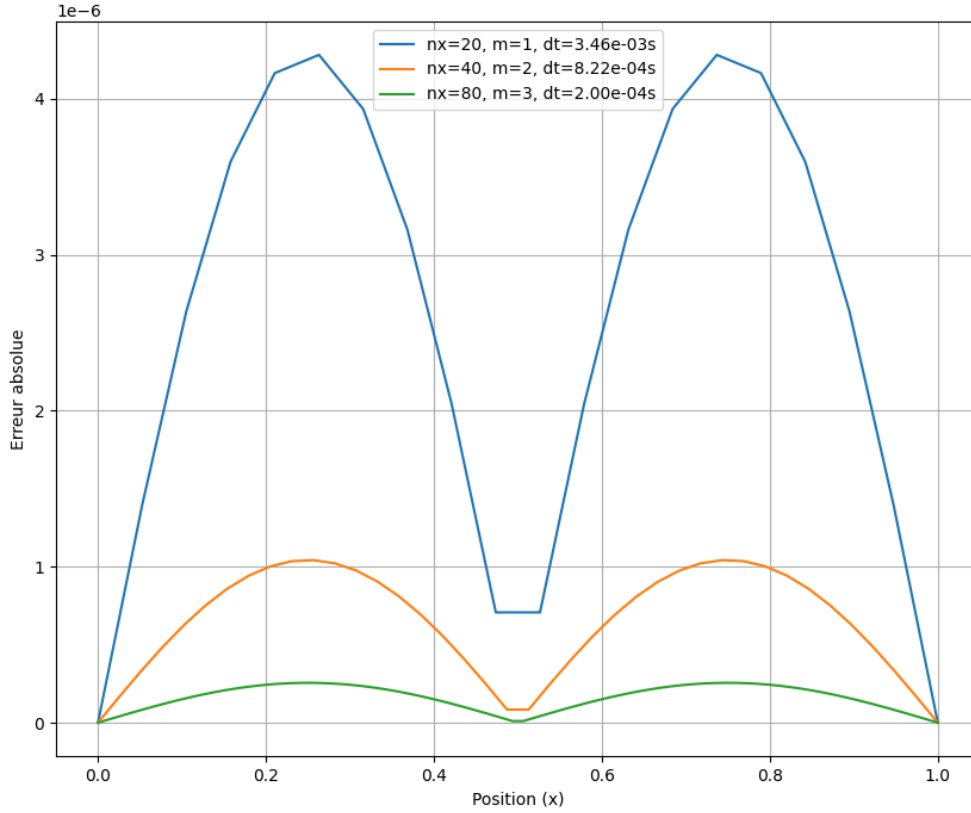


FIGURE 2 – Erreur en fonction de la position à $t=1s$

La figure 2 nous montre l'évolution de l'erreur en fonction de la position au temps $t=1$. Ce temps a été choisi parce qu'à $t=0$, l'erreur est nulle pour tout x puisque la condition initiale est fixée et est identique pour toutes les courbes. Comme nous pouvons l'imaginer, l'erreur à $m=1$ est la plus grande pour tout x suivie de $m=2$ et finalement $m=3$, ce qui nous permet de conclure que le nombre de points permet de diminuer l'écart entre la solution numérique et la solution analytique. Une chose intrigante à la figure 2 est que même si la température est censée être stable, nous observons un gradient de température sur la barre et un petit segment où l'erreur est constante à environ $x=0.5$.

1.4. Crank-Nicholson

1.4.1. Stabilité $\frac{\Delta t}{\Delta x^2}$

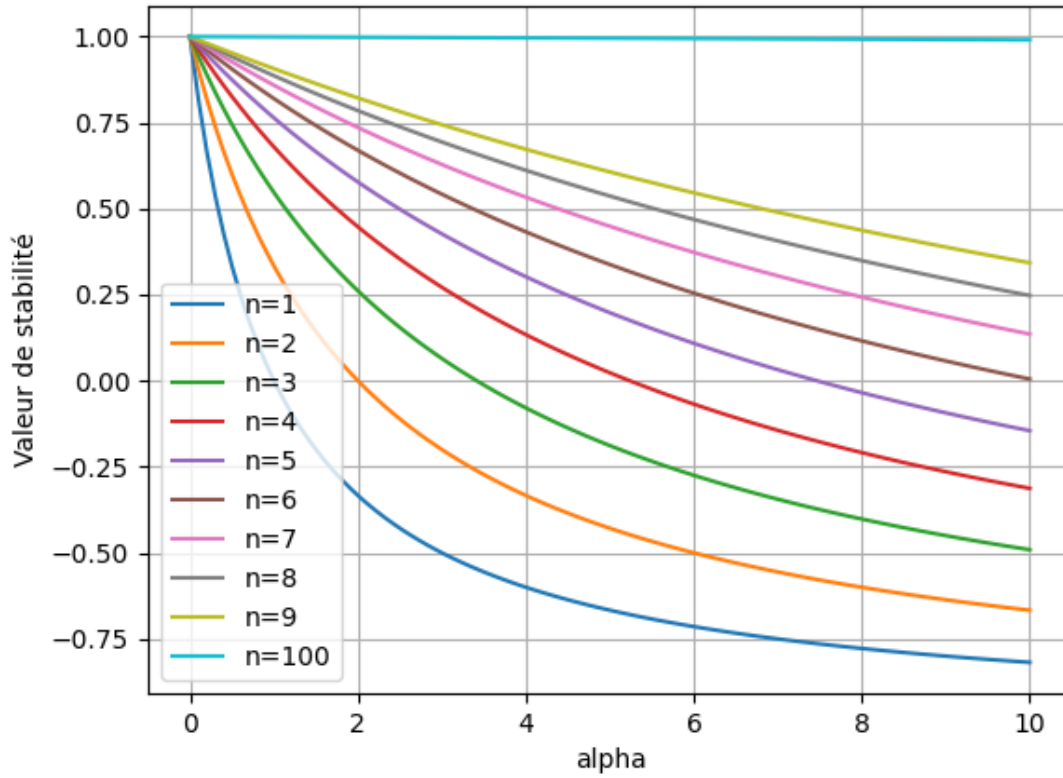


FIGURE 3 – Stabilité de la résolution par Crank-Nicholson en fonction du paramètre alpha

La figure ci-dessus nous montre la valeur de la stabilité en fonction d'alpha, qui est égal à $\frac{\Delta t}{\Delta x^2}$ pour plusieurs vecteurs propres, notés par n. Nous voyons que la stabilité tend vers 1 plus nous avançons dans les vecteurs propres sans jamais atteindre 1 sauf si alpha est égal à 0. Le code ci-dessous calcule la stabilité et nous avertit lorsque la stabilité atteint 1 :

```
def Crank_Nicolson_stabilite(alpha, l, n): #l est la longueur de la  
barre, n est un vecteur propre spécifique  
    theta = l*np.pi/(n+1)  
    num=2-alpha*(2-2*np.cos(theta))  
    denom=2+alpha*(2-2*np.cos(theta))  
    return num/denom
```

```

l=1.0
n=np.arange(1, 10) #On teste sur les 99 premiers vecteurs propres
n=np.append(n, 100) #On ajoute le 100eme vecteur propre pour
voir ce qui se passe a des valeurs plus elevees de n
alpha=np.linspace(1e-5, 10,100) #On teste sur des valeurs de alpha
allant de 0 a 10

stabilite=np.zeros((len(alpha), len(n)))
for i,N in enumerate(n):
    for j,a in enumerate(alpha):
        stab=Crank_Nicolson_stabilite(a, l,N)
        if stab >= 1:
            print(f'Instabilite detectee pour n={N} et alpha={a:.2f}
            avec une valeur de stabilite de {stab:.2f}')
        else:
            stabilite[j,i]=stab

```

Lorsque nous démarrons le code, nous n'obtenons aucun avertissement et seulement le graphique, mais l'avertissement apparaît lorsque nous posons la borne inférieure égale à zéro pour alpha. Nous pouvons alors en déduire que, pour tout alpha sauf zéro, la méthode par Crank-Nicholson est toujours stable.

1.4.2. Répétition du 3. avec Crank-Nicholson

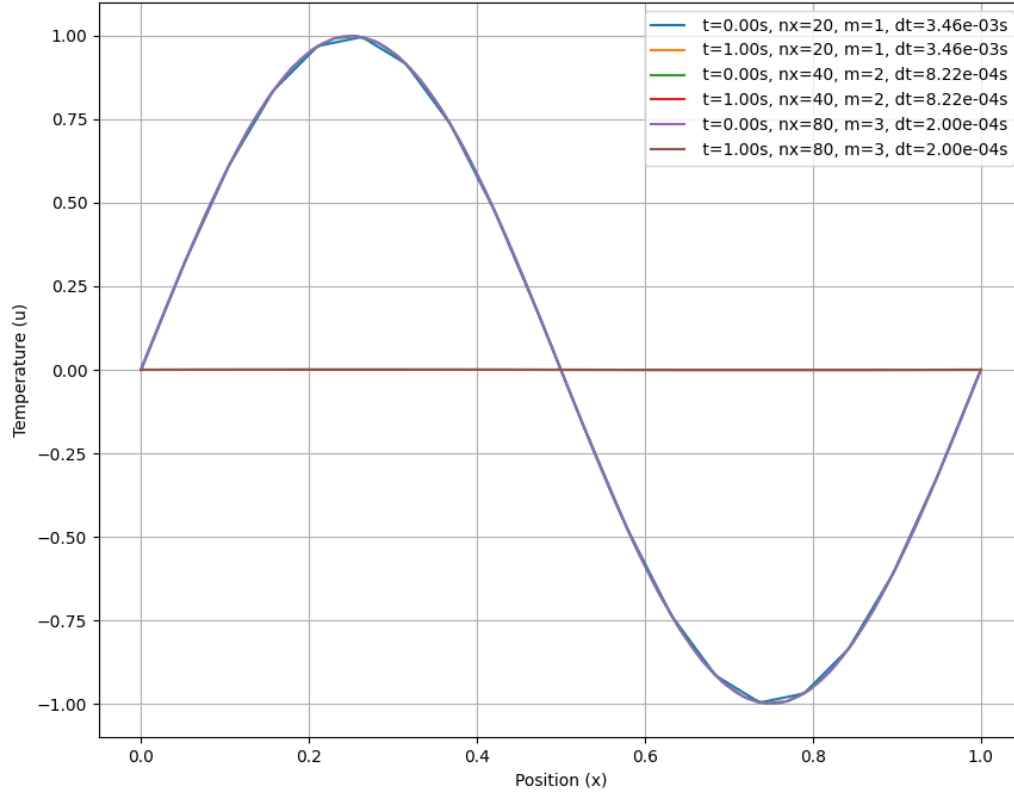


FIGURE 4 – Température dans la barre en fonction de la position à aux temps $t=0$ et à $t=1$ avec la méthode de Crank-Nicholson

La figure ci-dessus montre les mêmes conclusions que pour la résolution par la méthode explicite. La différence apparaît lorsque nous traçons l'évolution de l'erreur.

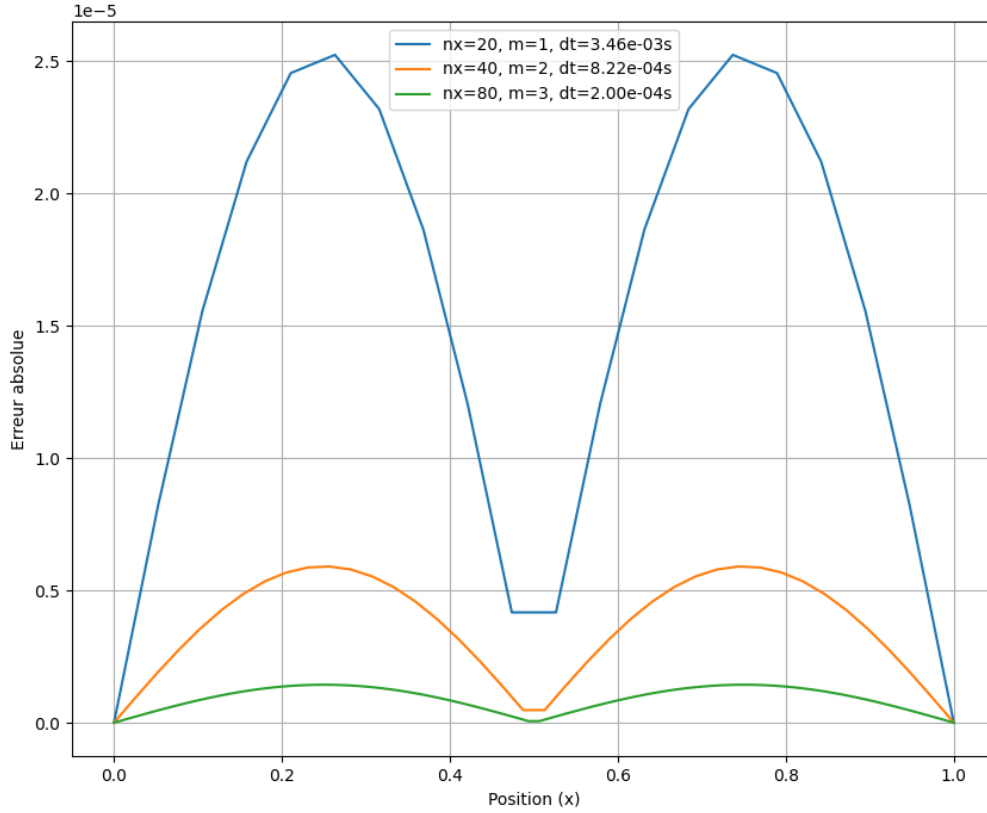


FIGURE 5 – Erreur en fonction de la fonction pour la méthode de Crank-Nicholson

Nous voyons le même comportement de la courbe d'erreur que pour la méthode d'Euler avancée. Cependant, l'erreur est plus grande pour tout x , ce qui nous permet de dire que la méthode de Crank-Nicholson être toujours stable, sauf qu'en échange, les erreurs sont plus grandes.

1.4.3. Coût d'une itération

Le coût itératif de Crank-Nicholson du même ordre de grandeur $O(n_x)$ qu'Euler explicite en terme du nombre de points à calculer, mais fait appel à la fonction plus souvent qu'Euler, puisqu'il y a une résolution par méthode implicite en plus d'une méthode explicite alors qu'Euler n'utilise qu'une méthode explicite. Donc, les deux méthodes possèdent un coût du même ordre de grandeur, mais la constante devant l'ordre de grandeur est plus haute que pour Euler parce qu'il y a une résolution implicite à faire en plus de la résolution explicite dans la méthode de Crank-Nicholson.

2. Intégration Monte Carlo

2.1. MC balistique

Valeur numérique	Erreur (%)	Nombre d'essais
0.0333581496	0.07	6

TABLE 1 – Intégrale numérique par MC balistique et son erreur en pourcentage

Le tableau 1 montre le résultat que nous obtenons lorsque nous faisons l'intégrale numérique de $\int_0^{10} e^{-3x} \sin^3(x) dx$ selon la méthode Monte-Carlo balistique¹. Pour définir la boîte, je calcule la valeur maximale et minimale de la fonction et, pour m'assurer d'être toujours dans la boîte, j'utilise des conditions *if* qui font en sorte que, lorsque le max est positif, on le multiplie par 1.05 et, s'il est négatif, on ramène la borne à 0. Pour la borne inférieure, si elle est positive, on la ramène à 0 et, si elle est négative, on la multiplie par 0.95. Voici à quoi ça ressemble en code :

```
x_boite=np.linspace(a,b,N)
y_boite=f(x_boite)
```

```
M_haut=y_boite.max()
```

```
if M_haut>0:
    M_haut*=1.05
```

```
elif M_haut<0:
    M_haut*=0
```

```
M_bas=y_boite.min()
```

```
if M_bas>0:
    M_bas*=0
```

```
elif M_bas<0:
    M_bas*=1.05
```

M_haut et M_bas représentent les bornes en y, donc il suffit de compléter la boîte en posant les bornes de l'intégrale comme borne sur l'axe x.

1. Voir Annexe A pour voir le code

2.2. Échantillonnage de la moyenne

Valeur numérique	Erreur (%)	Nombre d'essais
0.0332750927	0.17	1

TABLE 2 – Valeur numérique et erreur pour la méthode de la moyenne pondérée

Le tableau ci-dessus montre les résultats qu'on obtient lorsqu'on intègre numériquement $\int_0^{10} e^{-3x} \sin^3(x) dx$ avec la méthode de la moyenne pondérée².

2.3. Estimation de l'intégrale

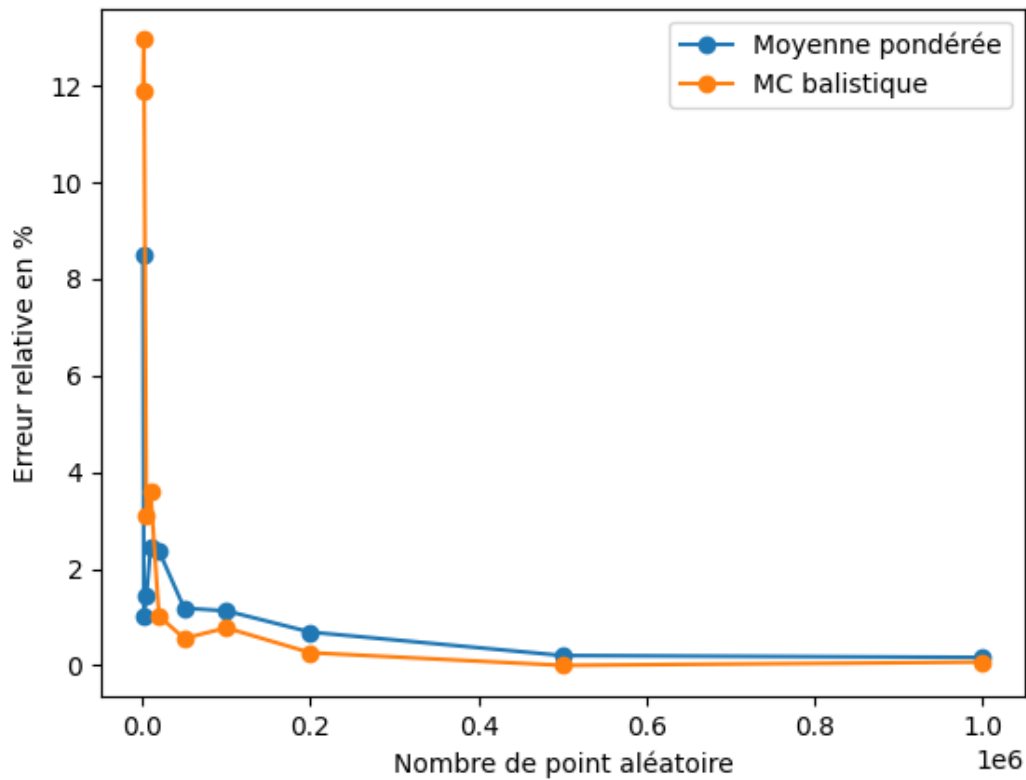


FIGURE 6 – Erreur relative en fonction du nombre de points aléatoires

La figure 6 montre qu'il y a une relation de puissance entre l'erreur et le nombre de points aléatoires, la théorie prédit une relation $\frac{1}{\sqrt{N}}$ entre l'erreur et le nombre de points. Nous voyons que nos points suivent la même tendance, mais la prochaine section confirmera si

2. Voir Annexe B pour voir le code

l'erreur suit vraiment la même tendance que prédit par la théorie. De plus, nous voyons à la figure 7 que la valeur de l'intégrale se stabilise vers la réponse analytique quand le nombre de points augmente. Cette tendance est due à la loi des grands nombres qui dit que la moyenne des valeurs d'une infinité de variables indépendantes tend vers l'espérance de la fonction.

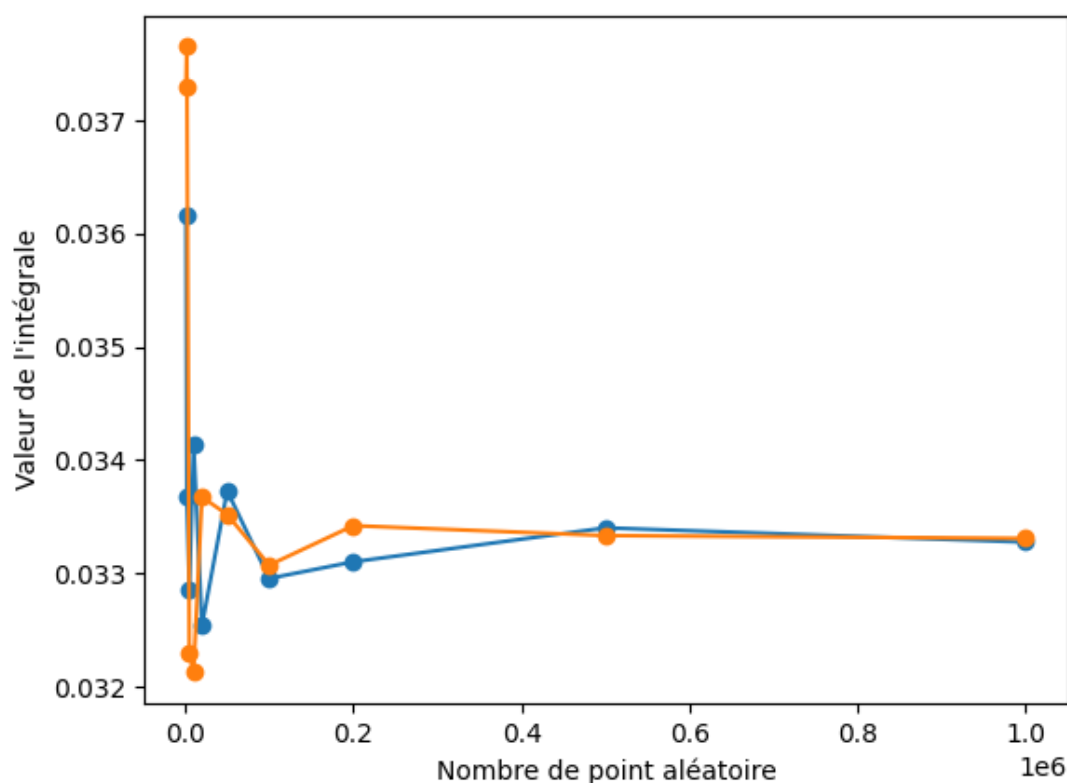


FIGURE 7 – Valeur de l'intégrale en fonction du nombre de points aléatoires

2.4. Erreur sans valeur théorique

Pour trouver l'erreur sans la valeur théorique, nous pouvons utiliser l'écart-type qui est donné par la variance. Il suffit alors d'appliquer la variance à nos formules d'intégration numérique pour trouver. Puisque les deux méthodes font appel à une moyenne multipliée par une constante pour obtenir le résultat, nous obtenons un écart-type de la forme³ :

$$\sigma(I) = C \sqrt{\frac{Var(I_i)}{N}}$$

3. Voir Annexe ?? voir pour la démarche

où C est une constante qui dépend de la méthode d'intégration numérique. Elle vaut $(b-a)$ pour la méthode de la moyenne pondérée et $(b-a)(M_{\text{haut}} - M_{\text{bas}})$ pour MC balistique. Voici son implémentation dans mon code :

```
def Int_Moyenne_Ponderee_erreur(f, params):
    a, b, N = params

    x = rng.uniform(a, b, N)
    y = f(x)

    variance = np.mean(y**2) - np.mean(y)**2

    erreur = (b - a) * np.sqrt(variance/N)

    return erreur
```

Pour le MC balistique, le calcul de l'écart-type est donné par :

```
fx = f(x)

points_in_posi = (fx >= 0) & (y >= 0) & (y < fx) # On recupere tous les
points qui sont dans la region positive a integrer en boolean
points_in_neg = (fx <= 0) & (y <= 0) & (y > fx)

points_in = points_in_posi.astype(int) - points_in_neg.astype(int) # On
recupere tous les point qui sont dans la region a integrer, en
prenant en compte les regions negatives. True=1 et False=0

variance = np.mean(points_in**2) - np.mean(points_in)**2

erreur = (b - a) * (M_haut - M_bas) * np.sqrt(variance/N)
```

Avec cet écart-type nous pouvons tracer la figure suivante :

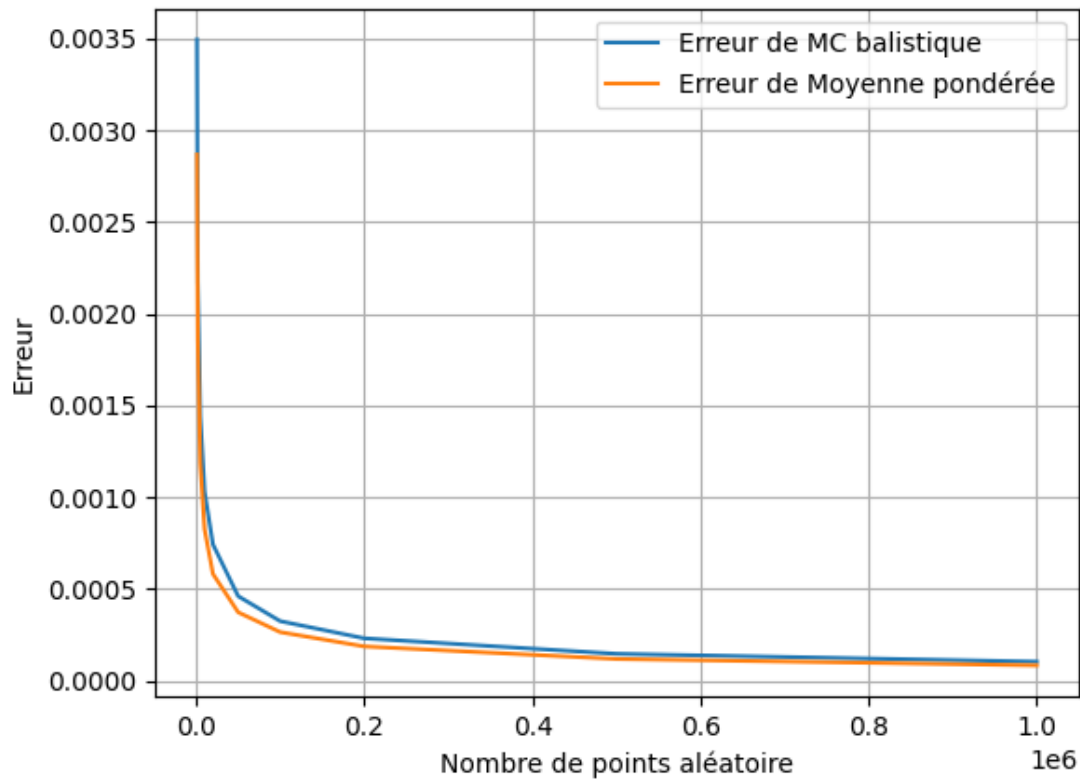


FIGURE 8 – Écart-type en fonction du nombre de points aléatoires

Les pentes des courbes ont été trouvées avec la fonction *polyfit* de numpy pour voir si nous avons la courbe prédite par la théorie. Voici l'implémentation du code :

```
parametres=[0,10,[1000,2000,5000,10000,20000,50000,100000,200000,500000,1000000]]
```

```
erreur_bal=[]
erreur_Moy=[]
```

```
fonction = lambda x: np.exp(-3*x)*(np.sin(x)**3) #Fonction qui prend x
en entree et retourne la valeur de la fonction a ce point
```

```
for N in parametres[3]:
    parametres_temp=[parametres[0],parametres[1],N]

    err_bal = MC_balistique_erreur(fonction, parametres_temp)
    erreur_bal.append(err_bal)
```

```
err_Moy=Int_Moyenne_Ponderee_erreur(fonction ,parametres_temp)
erreur_Moy.append(err_Moy)
```

```
pente_bal=np.polyfit(np.log(parametres[2]),np.log(erreur_bal),deg=1)
pente_Moy=np.polyfit(np.log(parametres[2]),np.log(erreur_Moy),deg=1)
```

Nous obtenons une pente de -0.5 pour les deux méthodes, ce qui nous permet de conclure que les courbes de la figure 6 concordent avec la théorie.

2.5. Intégrale 2D

Le graphique ci-dessous montre la valeur de l'intégrale avec les mêmes méthodes, mais avec une dimension de plus pour prendre en compte la deuxième intégrale :

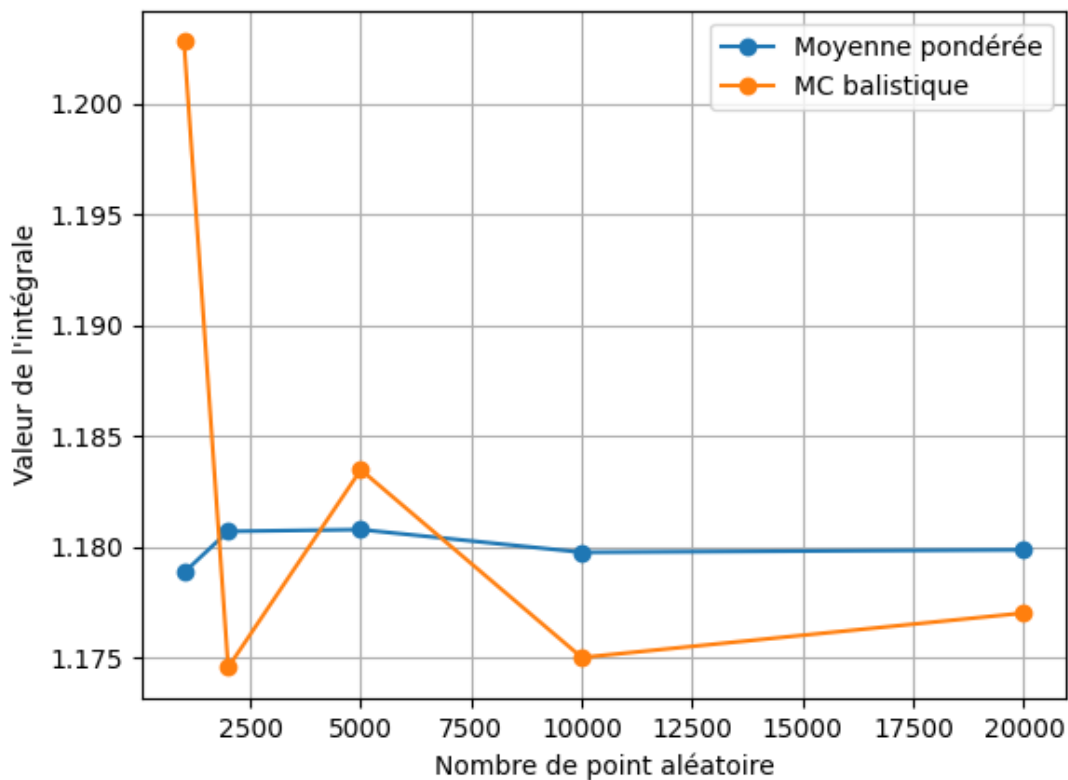


FIGURE 9 – Valeur de l'intégrale en fonction du nombre de points aléatoires

La figure ci-dessus montre une convergence vers le résultat analytique, mais la méthode MC balistique tend vers la valeur analytique, mais reste plus éloignée que la méthode de la moyenne pondérée. Voici un graphique avec l'erreur des deux méthodes :

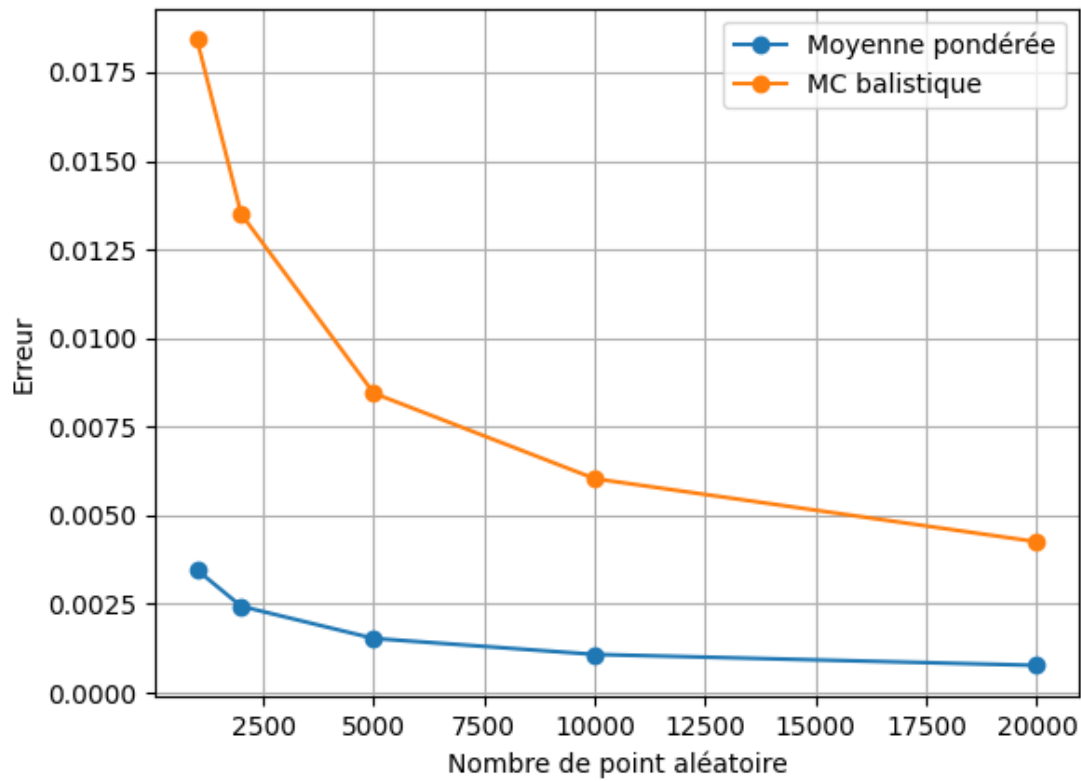


FIGURE 10 – Erreur en fonction du nombre de points aléatoires

La figure 10 montre bien la différence entre les deux méthodes lorsque nous intégrons sur deux variables. De plus, la méthode balistique n'est pas recommandée pour des intégrations avec plus qu'une variable, puisqu'avec deux variables, nous devons utiliser un réseau de taille N^2 pour contenir toutes les combinaisons de x et y possibles dans la boîte. Nous créons le réseau grâce à la ligne de code :

```
x_grille , y_grille=np.meshgrid(x_boite , y_boite)
```

où est-ce que x_boite et y_boite sont des tableaux de valeurs pour créer les boîtes.

2.6. Échantillonnage pondéré

Pour la méthode d'échantillonnage pondéré, nous avons utilisé les fonctions $g=\sqrt{1+x}$ et $g'=1+x$ pour l'importance de l'approximation pour le calcul de l'intégrale.

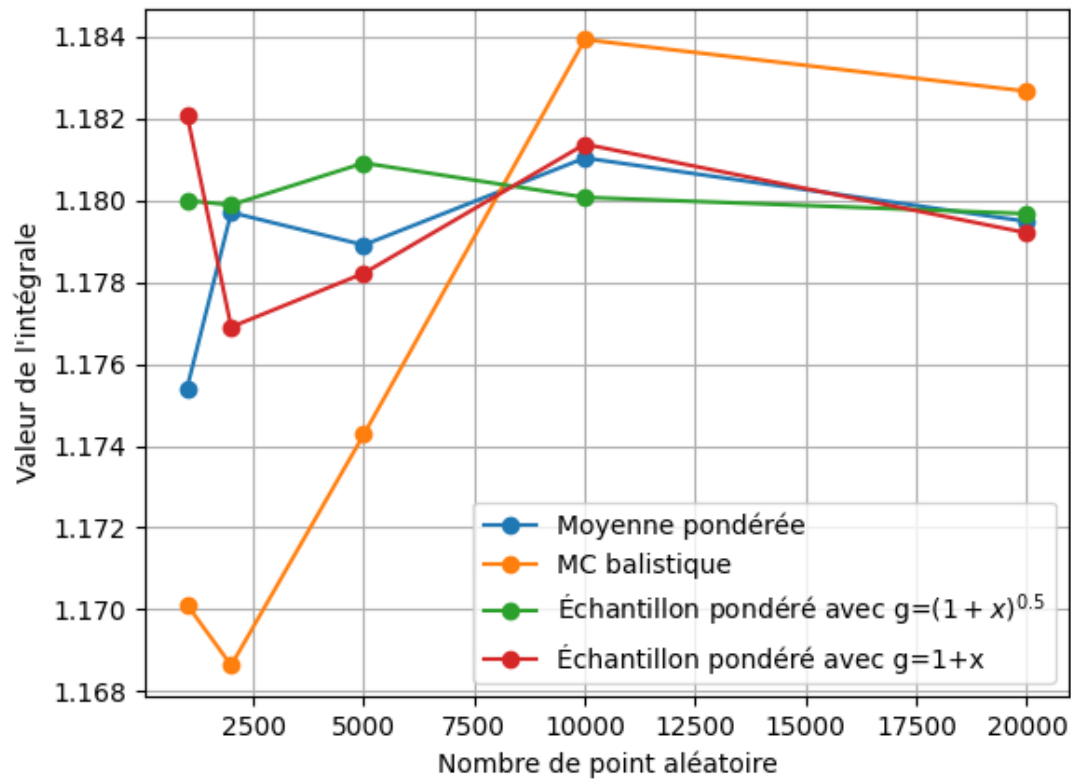


FIGURE 11 – Valeur de l'intégrale en fonction du nombre de points aléatoire pour 4 méthodes d'intégration numérique

La figure 11 montre que les 2 approximations sont très proches de la valeur analytique. Cependant, cela ne nous informe pas plus de la précision de la méthode d'échantillonnage pondéré. La figure ci-dessous montre l'erreur en fonction du nombre de points :

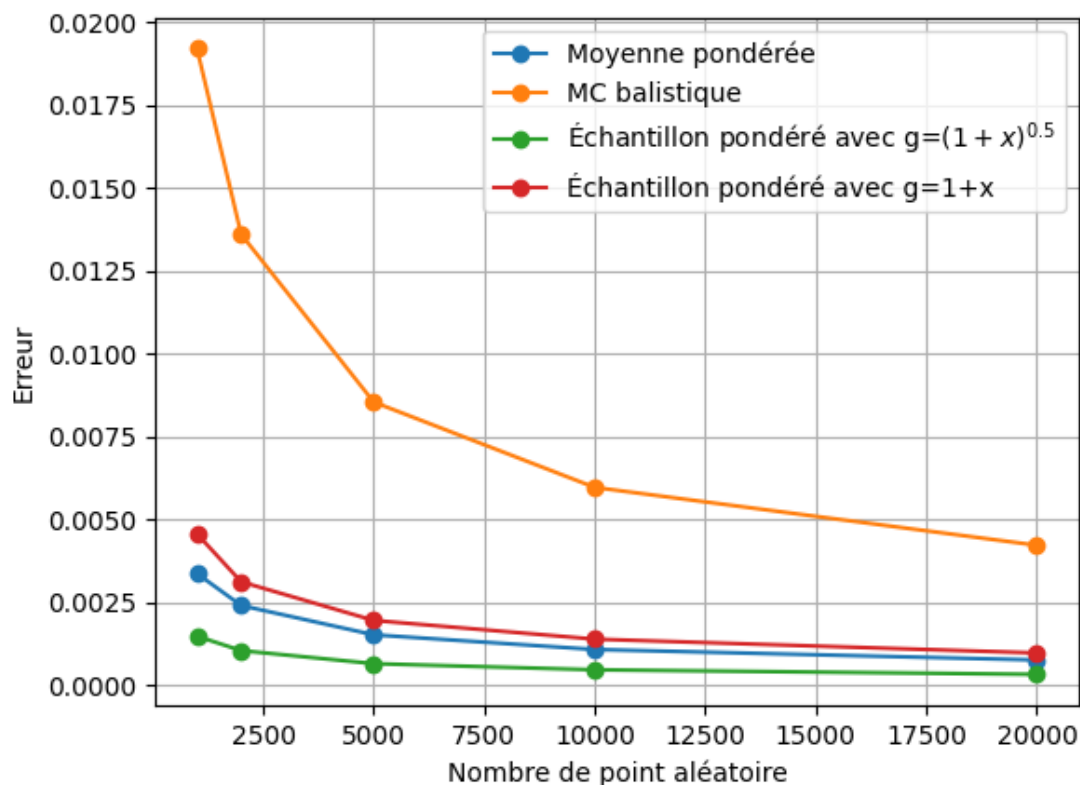


FIGURE 12 – Erreur en fonction du nombre de points aléatoire pour 4 méthodes d'intégration numérique

La figure 12 montre que la méthode d'échantillonnage pondérée est très précise, mais aussi que la forme de l'approximation g importe beaucoup sur la précision de la méthode. En effet, ajouter une racine carrée rend l'échantillonnage plus précis que la méthode des moyennes pondérées alors que si nous omettons la racine carrée, l'échantillonnage pondéré donne une erreur plus grande que la moyenne pondérée. Cette observation nous permet de conclure que plus notre équation d'approximation g est proche de l'équation à intégrer, plus sa précision sera grande.

3. Marche aléatoire

3.1. Générer un grand nombre de marcheurs aléatoire

Pour générer un grand nombre de marcheurs, j'ai ajouté une boucle *for* à la marche aléatoire et j'ai aussi créé des listes vides pour stocker les déplacements et positions de tous les marcheurs. Pour avoir un meilleur contrôle sur les paramètres de chaque marcheur, j'ai créé une fonction pour le pas et une autre pour la marche. Voici un

segment code pour mieux voir les changements :

```
#-----Recuperation des morceaux de code
des notes de cours sous forme de fonctions-----
def pas(rng):
    while True:
        x = rng.uniform(-1, 1)
        y = rng.uniform(-1, 1)
        global r2
        r2 = x*x + y*y
        if r2 <= 0.25:
            break

    r = np.sqrt(r2)
    x /= r
    y /= r
    return x, y

def marche_aleatoire_l(nombre_marcheurs, nombre_pas, rng):
    x_marcheurs = []
    y_marcheurs = []
    px_marcheurs = []
    py_marcheurs = []
    d2_marcheurs = []
    for marcheur in range(nombre_marcheurs):
        x = [0.0]
        y = [0.0]
        d2 = [0.0]
        px = []
        py = []
        # Boucle de marche aleatoire
        for i in range(1, nombre_pas): # 1..999 => total 1000 points
            dx, dy = pas(rng)
            px.append(dx)
            py.append(dy)

            xi = x[-1] + dx
            yi = y[-1] + dy
            d2i = xi*xi + yi*yi

            x.append(xi)
            y.append(yi)
            d2.append(d2i)
        x_marcheurs.append(x)
```

```

y_marcheurs.append(y)
px_marcheurs.append(px)
py_marcheurs.append(py)
d2_marcheurs.append(d2)
return x_marcheurs, y_marcheurs, px_marcheurs, py_marcheurs,
d2_marcheurs

```

3.2. Déplacement moyen et Variance

Pour le calcul du déplacement moyen, j'ai effectué une marche aléatoire, mais en ne récupérant que la liste contenant les déplacements de chaque marcheur. Par la suite, je récupère le dernier tableau qui donne la position finale et je calcule le déplacement moyen et la moyenne du déplacement au carré. Avec ces deux valeurs, je calcule la variance et l'écart-type. Je fais ce processus pour 10, 100, 1000 et 2000 marcheurs faisant tous 1000 pas. Voici le code pour mieux comprendre le fonctionnement :

```

for i, n in enumerate(nombre_marcheurs):
    __, __, __, __, d2=marche_aleatoire_1(n,nombre_pas, rng)
    d2_final = [d[-1] for d in d2]
    d_moyen[i] = np.mean(np.sqrt(d2_final))
    d2_moyen[i] = np.mean(d2_final)

var=d2_moyen - d_moyen**2
stdr=np.sqrt(var)/np.sqrt(nombre_marcheurs)
print(" Valeurs des moyennes: ", d_moyen, '+/-' ,stdr)
print(" Valeurs des variances: ", var)

```

Les figures 13 et 14 montrent les résultats obtenus et seront analysés à la prochaine section :

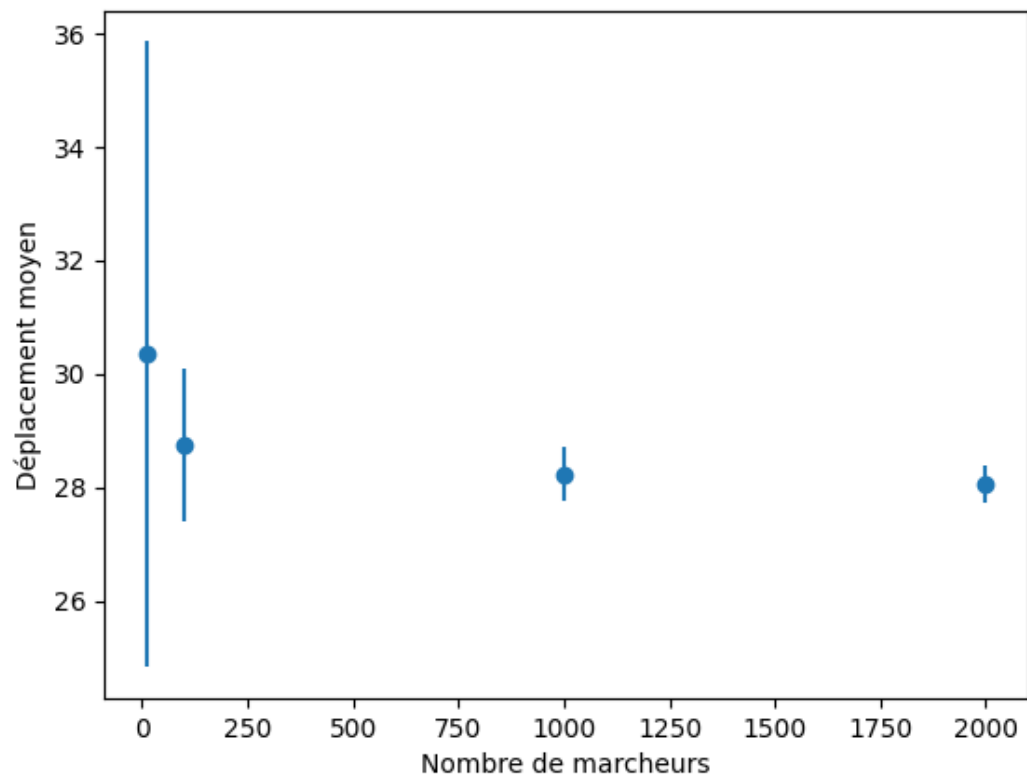


FIGURE 13 – Déplacement moyen en fonction du nombre de marcheurs faisant 1000 pas

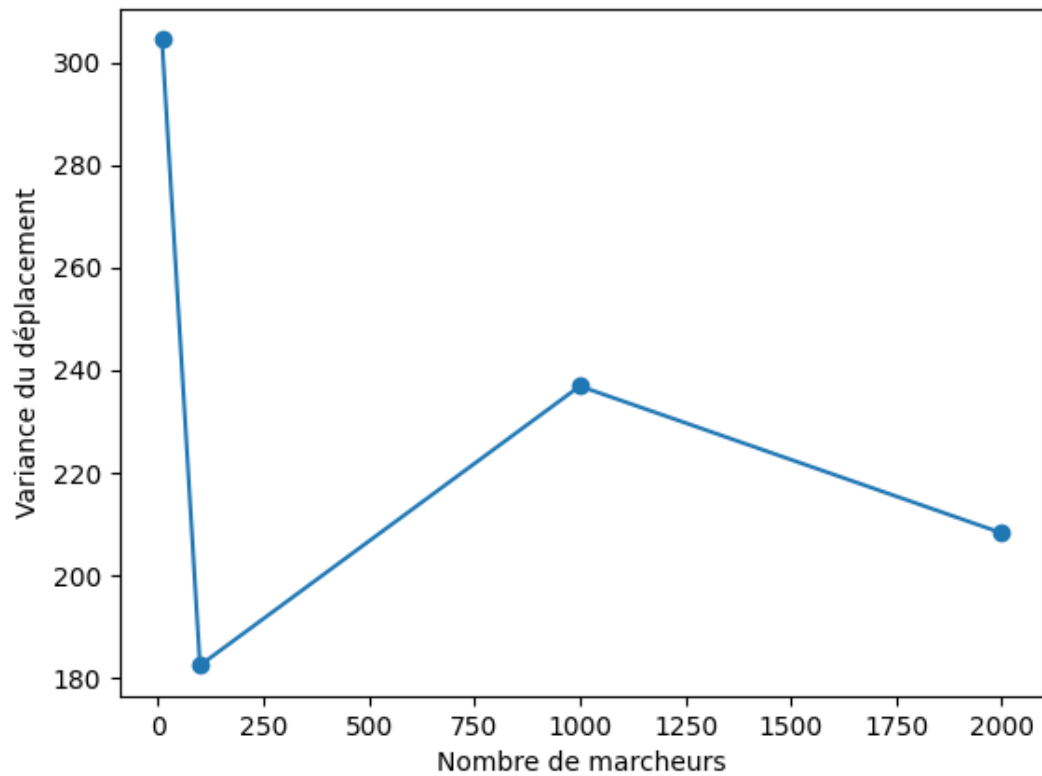


FIGURE 14 – Variance du déplacement moyen en fonction du nombre de marcheurs faisant 1000 pas

3.3. Concordance avec la théorie

Nombre de marcheurs	Déplacement moyen	Écart-type	Variance
10	30	6	304.41
100	28.8	1.4	182.53
1000	28.2	0.5	236.91
2000	28.3	0.3	208.30

TABLE 3 – Déplacement moyen, Écart-type et Variance selon le nombre de marcheurs

Selon la théorie, la marche aléatoire suit une distribution de Rayleigh dont la variance est donnée par :

$$Var(X) = \frac{4 - \pi}{2} \sigma^2 \quad (10)$$

et l'espérance est donnée par :

$$\mathbb{E}[X] = \sigma \sqrt{\frac{\pi}{2}} \quad (11)$$

avec $\sigma = \sqrt{\frac{n}{2}}$, nous obtenons alors :

$$\begin{aligned} Var(X) &= \frac{n(4 - \pi)}{4} \\ \mathbb{E}[X] &= \sigma \frac{\sqrt{n\pi}}{2} \end{aligned}$$

Sachant que n est le nombre de pas et est égal à 1000, la variance et l'espérance théorique est :

$$\begin{aligned} Var(X) &\approx 214.60 \\ \mathbb{E}[X] &\approx 28.0 \end{aligned}$$

En nous référant au tableau 3, nous voyons que le déplacement moyen est dans la région d'incertitude, ce qui nous permet de conclure que la valeur théorique possède 68% d'être dans l'intervalle. Pour la variance, nous voyons que nous sommes dans le bon ordre de grandeur, mais nous ne pouvons rien conclure de plus, puisque la simulation est très stochastique et qu'ajouter plus de marcheurs ajoute beaucoup de temps pour obtenir un résultat.

4. Voir Annexe D

A. Code du MC balistique

Le code a été réécrit pour enlever les accents que le package n'aime pas

```
def MC_balistique(f,f_theo,params): #f est la fonction, params est  
une liste de la forme [a,b,N] ou a et b sont les bornes de  
l'intervalle d'integration et N est le nombre de points a generer  
    a=params[0]  
    b=params[1]  
    N=params[2]  
  
    Int_theo=f_theo(b)-f_theo(a)  
  
    x_boite=np.linspace(a,b,N)  
    y_boite=f(x_boite)  
  
    M_haut=y_boite.max()  
  
    if M_haut>0:  
        M_haut*=1.05  
  
    elif M_haut<0:  
        M_haut*=0  
  
    M_bas=y_boite.min()  
  
    if M_bas>0:  
        M_bas*=0  
  
    elif M_bas<0:  
        M_bas*=1.05  
  
    err=100  
    essai=0  
    while err > 1:  
        x=rng.uniform(a,b,N)  
        y=rng.uniform(M_bas,M_haut,N)  
  
        points_in_posi=(f(x)>=0) & (y>=0) & (y<f(x))#On recupere  
tous les points qui sont dans la region positive a integrer  
        points_in_neg=(f(x)<=0) & (y<=0) & (y>f(x))  
  
        points_in=points_in_posi.astype(int)-points_in_neg.astype(int)  
#On recupere tous les points qui sont dans la region a integrer,
```

```

    en prenant en compte les regions negatives

    I=(b-a)*(M_haut-M_bas)*np.sum(points_in)/N #L'integrale est
    donnees par le pourcentage de points dans la region fois
    l'aire de la boite)

    ##### err=abs(1-(I/Int_theo))*100 #On calcule l'erreur en
    pourcentage
    essai+=1
    return I, err, essai

fonction = lambda x: np.exp(-3*x)*(np.sin(x)**3) #Fonction qui prend x
en entree et retourne la valeur de la fonction a ce point
f_theo=lambda x:np.exp(-3*x)*(-27*np.sin(x)+5*np.sin(3*x)-9*np.cos(x)
+5*np.cos(3*x))/120

parametres=[0,10,10000]
Int, yerr, essai=MC_balistique(fonction, f_theo, parametres)

print("L'integrale est=", Int)
print("L'erreur est=", yerr, "%")
print("Le nombre d'essais est=", essai)

```

B. Moyenne Pondérée

Ici aussi, le code a été retouché pour remplacer les accents.

```
def Int_Moyenne_Ponderee(f,f_theo,params):
    a,b,N=params

    Int_theo=f_theo(b)-f_theo(a)

    err=100
    essai=0

    while err > 1:
        x=rng.uniform(a,b,N)
        y=f(x)
        Int=np.mean(y)*(b-a)
        err=abs(1-(Int/Int_theo))*100
        essai+=1
    return Int,err,essai

fonction = lambda x: np.exp(-3*x)*(np.sin(x)**3) #Fonction qui prend
x en entree et retourne la valeur de la fonction a ce point
f_theo=lambda x:np.exp(-3*x)*(-27*np.sin(x)+5*np.sin(3*x)-9*np.cos(x)
+5*np.cos(3*x))/120

parametres=[0,10,10000]
Int,yerr,essai=Int_Moyenne_Ponderee(fonction,f_theo,parametres)

print("L'integrale est=",Int)
print("L'erreur est=",yerr,"%")
print("Le nombre d'essais est=",essai)
```

C. Écart-type

$$\sigma(I) = \sqrt{\text{Var}(I)}$$

$$= \sqrt{\text{Var}\left(C \mathbb{E}(I_i)\right)} \quad C = \text{constante} = \begin{cases} (b-a) \text{ pour Moyenne pondérée} \\ (b-a)(M_{\text{haut}} - M_{\text{bas}}) \text{ pour MC Balistique} \end{cases}$$

$$= C \sqrt{\text{Var}\left(\sum \frac{I_i}{N}\right)}$$

$$= C \sqrt{\frac{\sum \text{Var}(I_i)}{N^2}} \quad \left. \vphantom{\sum \text{Var}(I_i)} \right\} \text{chaque } I_i \text{ est indépendant}$$

$$= C \sqrt{\frac{N \text{Var}(I_i)}{N^2}}$$

$$= C \sqrt{\frac{\text{Var}(I_i)}{N}}$$

D. Démonstration de l'écart-type

En 2D, le déplacement d'un marcheur qui fait des pas de 1 peut être donné en radian sous la forme :

$$\Delta x = \cos(\theta)$$

$$\Delta y = \sin(\theta)$$

où θ est donné par une distribution uniforme $\theta \sim \mathcal{U}(0, 2\pi)$. Sachant que la moyenne de $\sin(n\theta)$ et $\cos(n\theta)$ est 0, la variance est alors donnée par :

$$Var(\Delta x) = \langle \cos^2(\theta) \rangle - \langle \cos(\theta) \rangle^2$$

$$Var(\Delta x) = \langle \cos^2(\theta) \rangle$$

$$Var(\Delta x) = \left\langle \frac{1 + \cos(2\theta)}{2} \right\rangle$$

$$\boxed{Var(\Delta x) = \frac{1}{2}}$$

$$Var(\Delta y) = \langle \sin^2(\theta) \rangle - \langle \sin(\theta) \rangle^2$$

$$Var(\Delta y) = \langle \sin^2(\theta) \rangle$$

$$Var(\Delta y) = \left\langle \frac{1 - \cos(2\theta)}{2} \right\rangle$$

$$\boxed{Var(\Delta y) = \frac{1}{2}}$$

Donc pour n pas, la variance et l'écart-type sont donnés par :

$$\boxed{Var(n\Delta x) = Var(n\Delta y) = \frac{n}{2}}$$

$$\boxed{\sigma = \sqrt{Var(n\Delta x)} = \sqrt{\frac{n}{2}}}$$